

Sistema Inteligente de Control de Acceso Híbrido

Del diseño a la implementación: una solución IoT Edge + Servidor

Murray Agustín (03255/1)
Polanis Iván Valentín (03266/5)
Savenia Manuel (03302/1)

Julio de 2026

Resumen

Se presenta la implementación de un sistema de control de acceso físico basado en RFID con una arquitectura **híbrida Edge + Servidor**. La lógica de decisión se distribuye entre un nodo perimetral (ESP32) y un servidor central (Node-RED), de modo que el acceso local sigue operativo aun ante la caída de la red o del broker. El nodo mantiene un **caché local** con tiempo de vida (TTL) e invalidación proactiva, mientras que el servidor actúa como **única fuente de verdad**, evaluando habilitación, expiración y franjas horarias sobre una base de datos relacional. La telemetría se registra en una base de series temporales y se visualiza en tableros; la administración se realiza en tiempo real mediante un bot de Telegram. El sistema fue montado en hardware físico y verificado de extremo a extremo. Este informe describe el contexto completo de la solución, la contrasta con la propuesta original, justifica las decisiones de arquitectura y analiza las ventajas y limitaciones de la tecnología empleada.

1 Contexto y objetivo

El problema abordado es el de un **control de acceso físico** (una puerta) que debe validar credenciales RFID y accionar una cerradura. Los sistemas comerciales tradicionales suelen adoptar uno de dos extremos: o bien concentran toda la lógica en un controlador local aislado (robusto pero difícil de administrar y auditar), o bien delegan cada decisión en un servidor central (administrable pero inutilizable si se pierde la conectividad). El objetivo de este trabajo fue construir una solución que combine lo mejor de ambos: **resiliencia local** con **administración y auditoría centralizadas**.

Sobre esa premisa se definieron los siguientes requisitos funcionales, todos implementados y verificados:

- Lectura de tarjetas RFID y accionamiento de una cerradura simulada (relé + LEDs).
- Decisión de acceso **sub-segundo** en el caso frecuente, sin depender de la red.
- Operación degradada tolerante a fallos de red (política de acceso definida).
- Padrón de usuarios administrable, con habilitación, expiración y franjas horarias por usuario.
- Bloqueo global instantáneo (*lockdown*) y baja inmediata de credenciales.
- Registro y visualización de todos los eventos de acceso para auditoría.

2 De la propuesta a la solución

La propuesta original planteó la arquitectura, el stack y la estrategia de caché. La implementación respetó ese diseño en su totalidad y, además, **amplió** varios puntos que en la propuesta quedaban abiertos. El siguiente cuadro resume la correspondencia:

Aspecto	Propuesta	Solución implementada
Arquitectura	Híbrida Edge + Servidor	Idéntica.
Caché en el edge	Caché local con TTL e invalidación	Implementada en LittleFS, persistente a reinicios, con purga periódica.
Validación central	Verifica estado en SQLite	Amplía a <code>enabled</code> + <code>expires_at</code> + <code>schedule</code> (franja horaria) como reglas evaluadas en el servidor.
Administración	Bot de Telegram	12 comandos: alta/baja, horarios, expiración, lockdown, TTL remoto, consultas y auditoría.
Telemetría	InfluxDB + Grafana	Idéntico. Dashboard de accesos provisto por <i>provisioning</i> .
Modelo de credencial	RFID genérico	MIFARE Classic vía RC522, usando el UID como identificador.

Tabla 1: Propuesta original vs. solución implementada.

3 Arquitectura de la solución

3.1 Visión general

El sistema se organiza en capas desacopladas que se comunican por MQTT. Cada componente cumple una única responsabilidad, lo que facilita reemplazarlo o escalarlo de forma independiente.

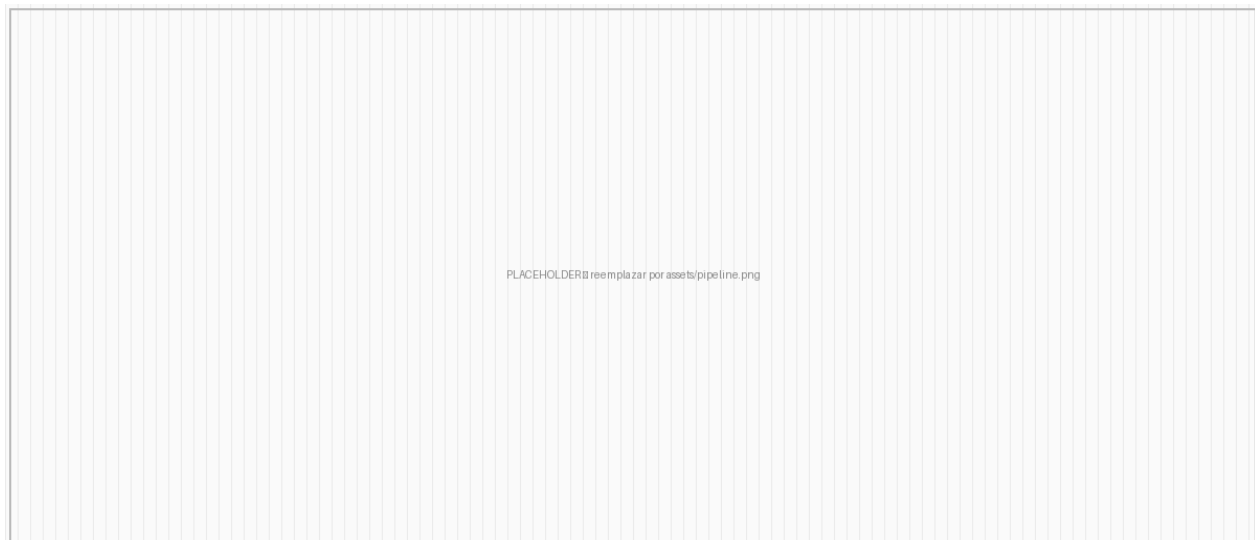


Figura 1: Arquitectura de la solución. El acoplamiento entre capas es únicamente por mensajes MQTT.

3.2 ¿Por qué una arquitectura híbrida?

La decisión central del diseño fue **dónde vive la lógica de acceso**. Se eligió distribuirla por tres razones:

Disponibilidad la apertura de una puerta es una función crítica que no puede depender de un enlace de red. Con el caché local, el nodo resuelve el caso frecuente (usuario ya conocido) sin tocar la red, y sigue funcionando ante cortes. El servidor deja de ser un punto único de falla para el uso cotidiano.

Latencia y tráfico consultar al servidor en cada lectura añade latencia y carga la red innecesariamente. El patrón *cache-aside* reduce el tráfico a solo los eventos nuevos (*cache miss*) y la telemetría, que se envía de forma asincrónica.

Administrabilidad y auditoría al mismo tiempo, mantener el **padrón y las reglas centralizados** evita tener que actualizar cada nodo a mano. El servidor es la única fuente de verdad; los nodos son cachés que se pueden invalidar. Así, una baja o un *lockdown* impactan al instante vía comando MQTT.

3.3 Elección del stack y su justificación

- **ESP32:** microcontrolador de bajo costo con WiFi integrado, doble núcleo y memoria suficiente para el caché y la cola. Ecosistema maduro (PlatformIO, Arduino).
- **MQTT / Mosquitto:** protocolo *pub/sub* liviano, ideal para redes inestables y dispositivos con recursos limitados. Ofrece QoS, mensajes *retained* y *Last Will* (LWT), que aprovechamos para detección de nodos caídos.
- **Node-RED:** permite construir el motor de reglas y la orquestación de forma rápida y visual, con integración nativa a MQTT, bases de datos y Telegram.
- **SQLite:** base relacional embebida, sin servidor, perfecta para un padrón de usuarios de esta escala. Consistencia transaccional para altas/bajas.
- **InfluxDB + Grafana:** combinación estándar para series temporales y su visualización; el registro de eventos es naturalmente temporal.
- **Docker:** define toda la infraestructura como código.

4 Funcionamiento de la solución

4.1 Inteligencia en el edge: caché y máquina de estados

Ante cada lectura de tarjeta, el firmware ejecuta una máquina de estados que prioriza la respuesta local y define explícitamente el comportamiento ante fallos:

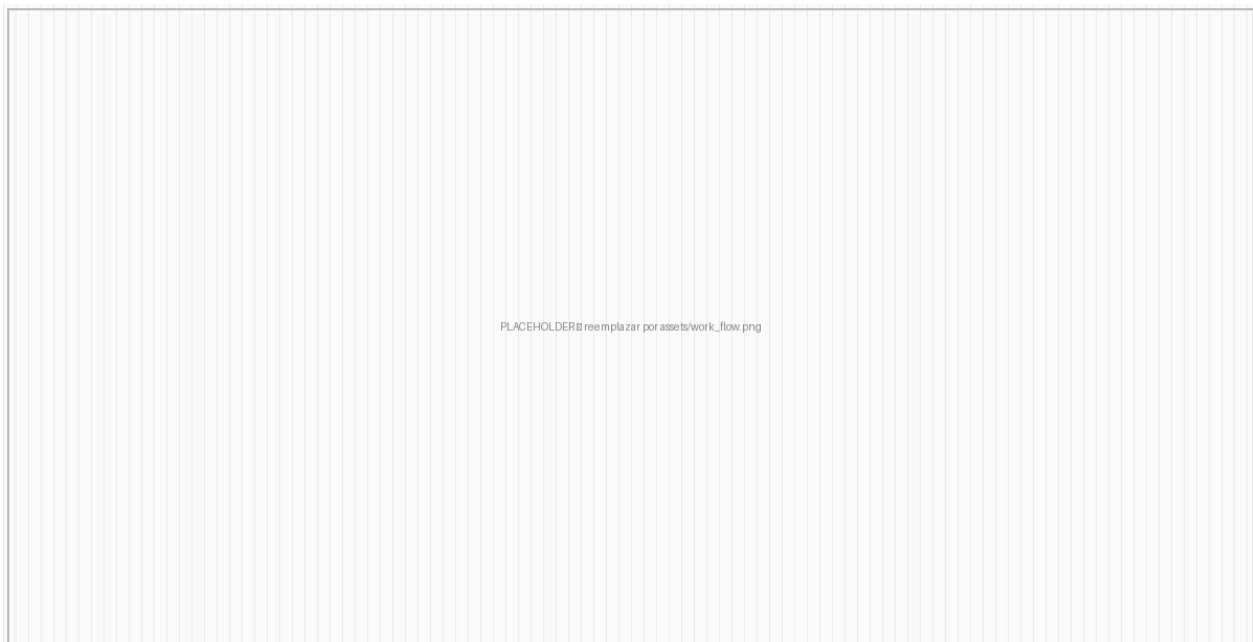


Figura 2: Flujo de decisión del ESP32 ante una lectura. Cada rama registra un evento con su **source** para auditoría.

Detalles de implementación relevantes:

- **Persistencia:** el caché (`cache.json`) y la cola de eventos (`queue.json`) se guardan en LittleFS, por lo que sobreviven a reinicios y cortes de energía.
- **TTL y expiración:** cada entrada del caché guarda un instante de expiración. El TTL por defecto es 24 h y es **configurable de forma remota** (`/ttl`). Una tarea periódica purga las entradas vencidas.
- **Sincronización horaria:** el nodo sincroniza la hora por NTP en segundo plano. Mientras no hay hora válida usa el *uptime*, y las entradas viejas quedan automáticamente marcadas como expiradas al comparar contra el tiempo Unix —un mecanismo de seguridad ante relojes no sincronizados.
- **Cola offline:** si al reportar un evento no hay red, este se encola y se drena automáticamente al reconectar, garantizando que **ningún acceso quede sin registrar**.
- **Invalidación proactiva:** el servidor puede ordenar borrar un UID (o todo el caché) por MQTT, de modo que una baja impacta de inmediato sin esperar al vencimiento del TTL.

4.2 Lógica central: el motor de reglas

El *cache miss* llega al servidor como un request MQTT. Node-RED consulta el padrón en SQLite y evalúa, en este orden: *lockdown* global → usuario existe → *enabled* → *expires_at* (no vencido) → *schedule* (día y franja horaria permitidos). Solo si todas las condiciones se cumplen responde *allowed=true* junto con un TTL —que además se acota inteligentemente al fin de la franja horaria o de la vigencia del usuario, para que el caché nunca autorice fuera de esos límites.

El padrón (*users*) almacena por usuario: *uid*, *name*, *enabled*, *expires_at*, *schedule* (JSON con días y horario) y marcas de tiempo. Una tabla *audit* registra toda acción administrativa (quién, qué y cuándo).

4.3 Registro, visualización y administración

Cada evento de acceso publicado por el nodo se persiste en **InfluxDB** y alimenta un dashboard de **Grafana** con el historial y las métricas de auditoría.

En paralelo, el **bot de Telegram** es la interfaz de administración del sistema: le da al operador el control completo del padrón y del estado de la puerta desde el teléfono, sin necesidad de acceder al servidor. A través de comandos simples permite dar de alta y de baja usuarios, habilitarlos o deshabilitarlos, configurar franjas horarias y fechas de expiración, activar el *lockdown* global, ajustar el TTL del caché de forma remota y consultar los últimos accesos o los datos de un usuario. Toda acción queda registrada en la tabla de auditoría junto con el administrador que la ejecutó. Además de la gestión, el bot funciona como canal de **alertas en tiempo real**: notifica ante un acceso denegado o cuando un nodo se da por caído (detectado a través de su *Last Will*). Como medida de seguridad, valida que el emisor sea un administrador autorizado antes de ejecutar cualquier comando.

Comando	Descripción y uso
/alta <uid> <nombre>	Da de alta un usuario en el padrón.
/baja <uid>	Elimina un usuario e invalida su caché en el nodo.
/habilitar <uid>	Rehabilita un usuario previamente deshabilitado.
/deshabilitar <uid>	Deshabilita temporalmente un usuario sin borrarlo.
/usuario <uid>	Muestra los datos de un usuario.
/usuarios	Lista todos los usuarios del padrón.
/horario <uid> <días> <desde> <hasta>	Fija la franja horaria permitida. Ej.: /horario A1B2 1,2,3,4,5 08:00 18:00 (lun-vie 08-18 h; 0=dom ... 6=sáb).
/expira <uid> <IS08601\ none>	Fija o quita la fecha de expiración. Ej.: /expira A1B2 2026-12-31T23:59:59Z.
/ultimos [n]	Muestra los últimos <i>n</i> eventos de acceso (10 por defecto).
/lockdown on\ off	Activa o desactiva el bloqueo global de accesos.
/ttl <segundos>	Cambia de forma remota el TTL del caché del nodo.
/ayuda	Lista todos los comandos disponibles.

Tabla 2: Comandos del bot de Telegram y su uso.

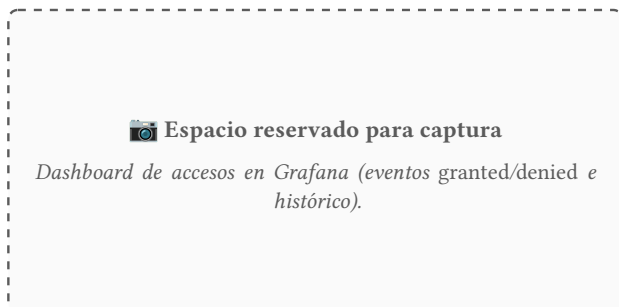


Figura 3: Dashboard de accesos en Grafana (eventos *granted/denied* e histórico).

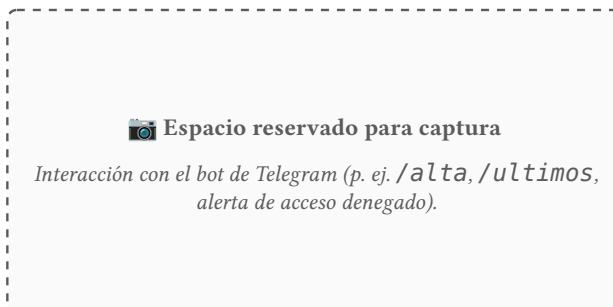


Figura 4: Interacción con el bot de Telegram (p. ej. /alta, /ultimos, alerta de acceso denegado).

 Espacio reservado para captura

Montaje físico: ESP32 + lector RC522 + relé y LEDs sobre protoboard, durante la prueba de extremo a extremo.

Figura 5: Montaje físico: ESP32 + lector RC522 + relé y LEDs sobre protoboard, durante la prueba de extremo a extremo.

4.4 Comunicaciones y seguridad de la mensajería

El broker exige autenticación (`allow_anonymous false`) y aplica una **ACL por tópico** que otorga a cada rol el mínimo privilegio: el ESP32 solo puede publicar en sus tópicos de evento/request/status y leer respuestas y comandos; Node-RED tiene acceso al espacio `access/#`. Los comandos y respuestas viajan con QoS 1 (entrega garantizada) y el estado del nodo es *retained*, de modo que un nuevo suscriptor conoce de inmediato si la puerta está en línea.

5 Ventajas y limitaciones de la tecnología

5.1 Ventajas

- **Resiliencia:** gracias al caché y a la cola offline, el sistema tolera cortes de red y de energía sin perder funcionalidad crítica ni registros.
- **Baja latencia y bajo tráfico:** la ruta rápida (*cache hit*) resuelve localmente en milisegundos; la red solo se usa para novedades y telemetría.
- **Administración centralizada:** altas, bajas, horarios, expiración, *lockdown* y hasta el TTL se gestionan de forma remota sin tocar el firmware.
- **Auditoría completa:** cada acceso y cada acción administrativa quedan registrados y son visualizables.
- **Desacoplamiento y portabilidad:** el patrón *pub/sub* y la contenerización permiten evolucionar cada pieza por separado y desplegar el *backend* en cualquier entorno.

5.2 Limitaciones

Seguridad del medio RFID el sistema utiliza tarjetas **MIFARE Classic** leídas con un módulo RC522. Este estándar está criptográficamente comprometido desde 2008: su cifrado propietario *Crypto1* fue quebrado y hoy una tarjeta se clona en segundos con hardware accesible. Además, en nuestra implementación se usa **únicamente el UID** como credencial, sin autenticación de sectores; y el UID es trivialmente falsificable con *magic cards* de UID reescribible.

Seguridad del transporte el MQTT del prototipo viaja sin TLS, por lo que las credenciales y los mensajes son observables en la red local. En producción correspondería habilitar `mqtts` (TLS) en Mosquitto.

Escalabilidad de SQLite es ideal para un único servidor y esta escala, pero su modelo de un solo escritor limita despliegues con muchos nodos concurrentes; ahí convendría PostgreSQL.

6 Conclusiones

Se cumplió el objetivo planteado en la propuesta: construir un control de acceso que sea **resiliente en el borde** y **administrable en el centro**. La arquitectura híbrida demostró ser acertada —la ruta rápida por caché entrega la respuesta inmediata que exige una puerta, mientras que el servidor concentra las reglas, la auditoría y la administración— y el sistema se validó de extremo a extremo sobre hardware real.

El desarrollo también amplió la propuesta donde esta quedaba abierta, transformando la validación central en un motor de reglas con habilitación, expiración y franjas horarias, y dotando al operador de una interfaz de administración completa por Telegram. La contenerización del *backend* hace que, aunque el desarrollo se realizó de manera local, **toda la infraestructura pueda trasladarse y desplegarse en la nube sin inconvenientes**, habilitando escenarios multi-puerta y acceso remoto a los tableros.

La principal lección de ingeniería es que **la elección de la tecnología de credencial define el techo de seguridad del sistema**: MIFARE Classic y el uso del UID resultaron cómodos y económicos para el prototipo, pero son el eslabón débil. Como líneas de trabajo futuro se identifican: migrar a credenciales con autenticación criptográfica (DESFire/NTAG o segundo factor), habilitar TLS en la mensajería, y evaluar una base de datos servidor para despliegues de mayor escala. En conjunto, la solución constituye una base sólida, funcional y extensible que materializa los principios de *edge computing* aplicados a un caso real.